MoM: Model Manifest

Specification 0.1.0 (draft)

2026-08-30

1 Abstract

MoM (MOdel Manifest) is a small YAML file, `mom.yaml`, that tells an AI tool which models to use and when to switch between them. You name a few models in plain words (a cheap one for everyday work, a smart one for hard problems), say which one to start with, and write switch rules anyone can read. Any tool that supports MoM reads the same file, follows the same rules, and can always tell you why it switched.

MoM is a model policy manifest: a user-authored switching policy the tool itself honors. It is not an ensemble or fan-out system and not a semantic router, and it is unrelated to vLLM's Mixture-of-Models.

A complete, useful file:

```yaml
mom: "0.1"

models:
  everyday:                  # fast and cheap, for normal work
    power: medium
  thinker:                   # the smartest model available
    power: max
    thinking: deep

start-with: everyday

switch:
  - when: planning
    use: thinker
  - when: stuck
    use: thinker
  - when: { spent-over: 5 }   # dollars
    use: everyday
```

The design bar: someone with no programming background can open another person's `mom.yaml`, understand every line, change a number, and share it.

2 Status of this document

- Specification version: 0.1.0 (draft)
- Date: 2026-08-30
- Manifest version string covered: `"0.1"`

- JSON Schema: `specs/mom.schema.json`, published alongside this document. If the schema and this text disagree, this text is authoritative.

This specification versions itself with Semantic Versioning. Backward incompatible changes to the file format bump the major version and the manifest `mom` version string together. Minor versions may add condition keywords, model entry keys, and top-level keys; section 8 defines how a tool built against an older minor version behaves when it meets them. The manifest string carries only major.minor (`"0.1"`, not `"0.1.0"`) because a patch release cannot change the format by definition.

3 Conformance language

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 (RFC 2119, RFC 8174) when, and only when, they appear in all capitals, as shown here.

Sections 3 through 9 are normative. The abstract, the examples in section 10, and the appendices are non-normative.

4 Terminology

Term	Definition
Tool	The AI harness that reads the file and talks to models: a coding agent, chat app, CLI, or editor plugin.
Manifest	The `mom.yaml` file.
Model entry	A named description of a model the user wants, such as `everyday` or `thinker`.
Catalog	A source of model metadata (context size, price, capabilities) used to resolve a model entry to a real model.
Accessible model	A model the user can actually reach: a configured provider, a valid key or login, and no tool-level block.
Rule	One entry in the `switch` list: a condition plus the model entry to use.
Condition	A named, testable check on the session, such as `stuck` or `spent-over`.
Turn	One user message and everything the tool does to answer it.
Session	One conversation from its first turn to its last, including any sub-work the tool spawns to answer within it.
Evaluation	One pass over the `switch` list to decide which model entry the next turn runs on.
Switch record	The audit entry a tool writes every time it changes models.
Demotion	A local, temporary decision to stop resolving an entry to a model that has failed in this user's own sessions.

5 File location and discovery

The manifest file name is `mom.yaml`, encoded UTF-8. A leading byte order mark MUST be tolerated. The file MUST be valid YAML under the YAML 1.2 core schema; authors MUST write booleans as `true` and `false` only (`yes`, `no`, `on`, and `off` are strings, never booleans).

A tool MUST look for the manifest in this order and use the first found:

1. `mom.yaml` in the project root
2. `.agents/mom.yaml` in the project root
3. a tool-defined personal location (RECOMMENDED: one documented path in the user's home configuration directory)

Exactly one manifest is active, loaded only from the paths above: there is no walking up parent directories. Files are not merged: a project manifest completely replaces a personal one, so what a file does never depends on another file the reader cannot see. When no manifest exists, the tool behaves as it does today; MoM is strictly opt-in. When a manifest exists but cannot be parsed, the tool MUST tell the user and MUST fall back to its default behavior rather than guessing.

A manifest configures the tool; it does not command the user. Section 7.4 defines how a user's explicit model choice overrides it.

6 Manifest format

Top-level keys:

Key	Type	Required	Meaning
`mom`	string	yes	Manifest format version. `"0.1"` for this specification.
`name`	string	no	Display name for the setup.
`models`	map	yes	Named model entries. At least one.
`start-with`	string	yes	Name of the model entry used when no rule matches. MUST name a key of `models`.
`switch`	list	no	Switch rules, checked in order.
`router`	map	no	Optional model-assisted fallback, off by default.
`x-*`	any	no	Tool-specific extras. Tools MUST ignore `x-` keys they do not recognize.

Unknown keys that do not start with `x-` make the manifest invalid under this version's schema. A validating tool SHOULD point at the misspelled key. Runtime handling of keys added by newer minor versions is defined in section 8.

6.1 `models`

Each entry describes what the user wants in plain words. The tool resolves the description against a catalog as defined in section 7.5. Entry names MUST match `[a-z0-9][a-z0-9-]{0,63}`.

Key	Values	Default	Meaning
`description`	string	none	What kind of work this entry is for, in plain words. Not used in resolution; it is what the router reads (6.4), and tools SHOULD show it when listing entries.
`power`	`low`, `medium`, `max`	`medium`	How capable the model must be. Defined in section 7.5.
`memory`	`normal`, `large`, `huge`, `vast`	`normal`	Minimum conversation size: at least 32k, 128k, 200k, or 1m tokens.
`thinking`	`none`, `some`, `deep`	`none`	The model must be able to reason this much before answering.
`sees-images`	boolean	`false`	Must be able to look at pictures.
`uses-tools`	boolean	`false`	Must be able to run tools.
`prefer`	list of strings	none	Exact model names tried in order first. A pin is a wish, never a requirement: the file MUST still resolve when a pinned model is unavailable.
`settings`	map	none	Knobs passed through to the model: `effort`, `temperature`, `max_tokens`, plus `x-` namespaced extras.

`thinking` is a requirement on the model; `settings.effort` is how hard to run it. When `thinking` is `some` or `deep` and `settings.effort` is absent, the tool SHOULD run the model at a matching effort rather than its floor, so the plain form does what it looks like it does.

Valid:

```
models:
  marathon:
    power: medium
    memory: huge
    prefer:
      - anthropic/claude-sonnet-5
```

Invalid (power value not in the list; exact IDs go under `prefer`):

```
models:
  marathon:
    power: claude-opus-5
```

6.2 `switch`

A rule is a condition plus the model entry to use.

Key	Type	Required	Meaning
`when`	condition or list of conditions	yes	The condition. A list means any of them.
`use`	string	yes	Model entry name to switch to. MUST name a key of `models`.
`hold`	integer	no, default 3	Keep this choice for at least N turns, even after the condition clears.

A condition is written one of two ways:

- a bare keyword, using its default: `stuck`
- a map with one keyword and its parameter: `{ stuck:  5 }`

In a list, a condition the tool cannot evaluate (an unknown keyword, or a condition made inert by section 6.3) is dropped from the list; the rule matches if any remaining condition holds. A rule whose conditions are all inert is itself inert.

6.3 Conditions

The condition keywords, their parameters, and their exact meanings. This table is the normative definition; every conforming tool MUST evaluate each keyword identically.

Keyword	Parameter	Default	True when
`planning`	mode name (string)	the tool's planning or read-only mode	the tool is in that mode
`stuck`	failed steps (integer)	`3`	the parameter count of tool steps have failed in a row on the current model
`looping`	none	none	the current model is repeating itself or producing empty output
`chat-full`	percent (number)	`70`	the conversation occupies more than that percent of the context window of the model this evaluation would otherwise select, measured by the procedure in section 7.1
`spent-over`	dollars (number)	none, parameter REQUIRED	this session's estimated cost has passed that amount
`tokens-over`	tokens (integer)	none, parameter REQUIRED	this session has used more than that many tokens
`turn-over`	turns (integer)	none, parameter REQUIRED	more than that many user turns have passed
`model-down`	none	none	the current model is unavailable, erroring, or refusing requests under load

`spent-over`, `tokens-over`, and `turn-over` are the spending conditions; a rule whose matched condition is one of them is a spending rule. `looping` and `model-down` are the emergency conditions (section 7.2); a rule matched through one of them is an emergency rule.

Measurement rules:

- `stuck` counts failures on the current model only. Its counters MUST reset when the model changes, so a fresh model never inherits the last one's failures.
- `looping` detection (what counts as repetition or emptiness) is tool-defined; when a tool acts on the detection is defined here and in section 7.2.
- `chat-full` is deliberately not measured against the current model: measured that way, switching to a bigger model empties the percentage, the rule clears, the session drops back to the small model, fills it, and switches again, forever. Measured against the model the evaluation would otherwise select, the rule keeps matching exactly as long as the conversation genuinely would not fit comfortably, and clears when it would. Section 7.1 defines the measurement procedure.
- The spending conditions never decrease. A matched spending rule therefore keeps matching for the rest of the session, and because earlier rules win, authors SHOULD place spending rules last. This is by design: a budget is a fact about the whole session, not a passing state.
- Cost estimates MUST be computed from catalog prices and MUST cover every model call in the session: the main loop, router calls, and any sub-work. The minimum cost model prices input, output, cache, and reasoning tokens whenever the catalog carries a price for that component; a component the catalog does not price is estimated as zero, and the tool MUST say once which components are missing. When a tool cannot estimate cost at all, `spent-over` rules are inert and the tool MUST say so once.

Valid:

```
switch:
  - when: [stuck, looping, model-down]
    use: thinker
  - when: { chat-full: 85 }
    use: marathon
  - when: { spent-over: 2.50 }
    use: everyday
```

Invalid (free text is not a condition; `spent-over` needs its parameter):

```
switch:
  - when: the task looks hard
    use: thinker
  - when: spent-over
    use: everyday
```

Tools MAY define extra conditions under an `x-` prefix (for example `x-mytool-in-review`). A condition keyword a tool does not recognize makes that condition inert as defined in 6.2: the tool MUST NOT reject the file, SHOULD report the unknown keyword once, and MUST keep evaluating everything else. Semantic judgment ("this looks like a design task") is deliberately not a condition; that is the router's job.

6.4 `router`

When enabled and no rule matched, the tool MAY ask a model to pick the entry for this turn instead of using `start-with`.

```
router:
  enabled: true              # default false
  power: low                 # the picking model should be cheap
  prefer:                    # optional pin, resolved like an entry's prefer
    - zai/glm-5.3-flash
```

The router contract:

- The router model is resolved like a model entry with the given `power` and optional `prefer` list. Authors SHOULD pin `prefer` when the tool reaches an aggregator: bare `power: low` resolves to the cheapest accessible model in the whole catalog, which is rarely the intent.
- Its input is at most: the entry names and their manifest descriptions, and the opening state of the turn, which is the user's message text for this turn and nothing more: no prior history, no tool output, no system prompt. Its output MUST be the closed JSON shape `{"use": "<entry-name>"}` naming a declared entry; anything else, any error, and any timeout MUST fall back to `start-with`.
- The router MUST NOT override a matched rule, MUST NOT introduce models outside `models`, and its calls count toward `spent-over` and `tokens-over`.
- A pick that changes the entry is a switch like any other: it produces a switch record and holds (default 3) so the router is not consulted again every turn. A pick that keeps the current entry is not a switch and MUST NOT hold: holding it would pin the session on the entry it is already on and stop the next turn from being judged at all.
- Tools SHOULD record every consultation and its outcome (the pick, or the error that caused the fallback), since a pick that keeps the current entry produces no switch record and is otherwise invisible.

7 Runtime behavior

7.1 The evaluation model

Before each user turn, the tool runs one evaluation:

1. Resolve every model entry to a concrete model, or mark it unresolvable (7.5). A rule whose `use` entry is unresolvable is skipped.
2. Compute the conditions.
3. Walk the emergency rules (rules matched through `looping` or `model-down`) top to bottom; the first match selects its `use` entry. Stop.
4. Otherwise walk the remaining rules top to bottom; the first match selects its `use` entry. Stop. `chat-full` conditions are measured by the procedure below.
5. If no rule matches: the entry selected by a still-running `hold` (7.2) if any, else the router's pick if enabled (6.4), else `start-with`.

Emergency rules are walked first so that getting unstuck beats every other concern, including a spending rule placed above the emergency rule: a dead model is worth the money. For every other pair of rules, order decides, and nothing else does.

Measuring `chat-full`: run steps 4 and 5 with every rule containing a `chat-full` condition removed, and resolve that selection. That is the model this evaluation would otherwise select. Measure the conversation against its context window, evaluate every `chat-full` condition against that one measurement, then run step 4 with those rules included. If the selection changes, check it against the safety limits (7.3) once and stop; the measurement is not recomputed.

Evaluation is stateless apart from `hold`, demotions (7.6), and the `stuck` counters. In particular, when a rule stops matching, its effect ends: rules describe states the session is in, not one-way transitions. A `planning` rule puts the session on `thinker` exactly while the tool is in planning mode (plus `hold`), then the session falls back to `start-with` on its own. Without this, every mode rule would be a one-way ratchet onto the expensive model.

A worked trace of the abstract's file (hold 3 everywhere):

Turn	State	Evaluation	Model
1	nothing special	no match, start-with	everyday
2	user enters plan mode	`planning` matches, hold set to 3	thinker
3	plan accepted, executing	no match, thinker held (2 left)	thinker
4	executing	no match, thinker held (1 left)	thinker
5	executing	hold expired, no match	everyday
9	3 tool steps failed	`stuck` matches	thinker, counters reset
12	recovered	hold expired, no match	everyday
20	$5.20 spent	`spent-over: 5` matches	everyday (already there)
21	3 tool steps failed	`stuck` matches first	thinker: order decides

Turn 21 works because the `stuck` rule is written above the spending rule; `stuck` is not an emergency, so if the author had put the spending rule first, the budget would win. A second trace, with the spending rule deliberately placed first:

```
switch:
  - when: { spent-over: 5 }
    use: everyday
  - when: stuck
    use: thinker
  - when: looping
    use: thinker
```

Turn	State	Evaluation	Model
20	$5.20 spent	`spent-over` matches first	everyday
21	3 tool steps failed	`spent-over` still matches first	everyday: order decides
22	model repeats itself	`looping` is an emergency, walked first	thinker: emergency beats budget

The switch is the whole point of the file, so one more rule: switching models MUST NOT lose the conversation. The full session history moves to the new model, translated however the tool already translates histories between providers.

7.2 Holds and emergencies

A rule's choice MUST last at least `hold` turns (default 3), counting the turn it takes effect as the first; every turn the rule matches restarts the count. Concretely: when a rule matches, its hold counter is set to `hold`. On an evaluation where it does not match, the counter goes down by one, and while the counter is above zero the held entry is used when no rule matches (7.1 step 5). This stops flip-flopping when a condition hovers at its edge. During a hold, evaluation still runs, and a matching rule wins over the held choice; the hold only outlasts the silence after the condition clears, it never outvotes a rule. A hold on a spending rule is harmless but useless: spending conditions never clear.

Two conditions are emergencies: `model-down` and `looping`. Their rules are walked before all others (7.1 step 3), and their choice overrides any running hold.

Emergencies MAY also act mid-turn, because the alternative is a dead turn. Exactly two events permit a mid-turn evaluation: the provider returns an error or refusal that makes `model-down` true, or the tool's `looping` detection fires. A mid-turn evaluation runs only steps 1 through 3 of 7.1 (the emergency walk). A mid-turn switch counts against the two-switch turn budget (7.3), resets the `stuck` counters like any switch, and the switched-to rule's hold count starts at the next user turn.

7.3 Safety limits on switching

- A tool MUST NOT switch to a model whose context window cannot hold the current conversation. A candidate that fails this check is skipped for this evaluation, falling through to the next resolvable choice.
- A tool SHOULD NOT let one evaluation switch more than once, and MUST NOT switch more than twice in one turn including emergencies. A session that would ping-pong faster than that has a broken manifest, and the tool SHOULD say which rules are fighting.

7.4 The user always wins

When the user explicitly picks a model in the tool (a `/model` command, a dropdown, a flag), that choice MUST override the manifest from that point on, and the tool MUST NOT silently switch away from it. The tool SHOULD tell the user that the manifest is suspended and how to resume it. Emergencies MAY still propose a switch, but as a question, not an action. A manifest automates the user's own policy; the moment the user speaks, the file is the junior partner.

7.5 Resolving model entries

Models differ not only in what they can do but in how well they do it: two models with `uses-tools: true` can be worlds apart at actually calling tools or following a skill. The protocol handles this with three sources of knowledge, consulted in order, each living where it can stay true:

1. **What the file wishes.** The `prefer` pins: the user's own experience of which models are good, shared with the file.
2. **What the market knows.** The catalog: price, context size, capability flags, and quality data such as tool-use benchmark scores. It updates globally without touching anyone's manifest.

3. **What this user has seen.** The switch records from 7.7: which models actually got stuck or died in this user's own sessions. Local evidence, defined in 7.6.

A catalog MUST provide, per model: context window size, prices, and whether the model reasons, sees images, and calls tools. models.dev is the RECOMMENDED default catalog; a tool MAY substitute or vendor its own with the same fields, and MUST keep working offline from its most recent snapshot.

Before resolving, a tool MUST intersect every entry's stated requirements with its own operational requirements: a tool whose main loop calls tools never resolves any entry to a model that cannot call tools, whatever the manifest says. The manifest states the user's floors; the tool adds its own. This keeps a field's meaning identical on every host.

Resolution order for an entry: each `prefer` pin in order, skipping inaccessible and demoted models; then the cheapest accessible, non-demoted catalog model satisfying every requirement, chosen through the `power` mapping below.

`power` is defined relative to what the user can access, so the same file resolves sensibly for a hobbyist with one key and a team with ten. The candidates are the accessible, non-demoted models satisfying the entry's other requirements, ranked best to worst by the catalog's quality score when it carries one, otherwise by blended per-token price as a proxy. Then:

- `max`: the top of the ranking.
- `low`: the cheapest candidate.
- `medium`: the cheapest candidate in the upper two-thirds of the ranking; never the same pick as `low` when more than one candidate exists.

A tool MUST document which ranking inputs it uses (quality score or price proxy, and from which catalog), and `mom check` style commands SHOULD show what every entry resolves to right now, so "which model will I actually get" is never a mystery.

Capability flags are floors, not grades. When the catalog carries quality data for a capability (for example tool-use benchmark scores), a tool SHOULD prefer the cheapest model that is dependable at it over one that merely has the flag. Quality is never a manifest field; it enters only through the three sources above, so a shared file never carries a claim that goes stale.

When nothing satisfies an entry, the tool MUST tell the user which requirement failed and fall back to `start-with`, or to its own default when `start-with` itself cannot resolve.

7.6 Learning from switches

The switch record is not only an explanation; it is evidence, and it closes the protocol's loop: describe, resolve, observe, resolve better.

When the same resolved model triggers `stuck`, `looping`, or `model-down` twice within a session under the same model entry, a Level 1 tool SHOULD demote it: skip it when resolving that entry for the rest of the session, unless it is the only model that satisfies the entry. A tool MAY persist demotions per project so a model that fumbles tools in this codebase stops being chosen in it; persisted demotions SHOULD expire (RECOMMENDED: 30 days or on catalog update) because models and providers improve.

Demotion is local. A tool MUST NOT write demotions into the manifest and MUST NOT share them; the manifest carries wishes, the catalog carries the market, and evidence stays with the user who observed it. A demotion MUST be visible in the switch record it results from ("skipping model X for thinker: got stuck twice this session"), so this loop is as explainable as every other switch. A tool SHOULD let the user list and clear active

demotions (a `mom status` style command), so a demotion is never mistaken for a broken manifest.

This is how the protocol answers per-model skill differences in the large: nothing predicts which model fumbles tools, the session proves it, and resolution stops choosing it. Good tool callers win locally and automatically, with no quality field to author or keep current.

7.7 The switch record

Every switch MUST be recorded with: which rule fired (its position, or the literal `router`, `start-with`, `emergency`, or `user`), the condition values at that moment, the model entry switched from and to, the actual model resolved, and any model skipped by demotion or the safety limits. Tools SHOULD expose the records in plain language ("switched to thinker: stuck, 3 failed steps in a row"). "Why did it switch" always has an answer, and "why did it pick that model" does too.

7.8 Scope within a session

The manifest governs the tool's main loop. When the tool spawns sub-work to answer a turn (sub-agents, parallel helpers), it MAY apply the manifest to them or keep its own scheme; either way their usage MUST count toward `spent-over` and `tokens-over`, because the user pays for the session, not the loop.

8 Conformance

- **Level 0.** Parse the manifest, resolve and use `start-with`, ignore `switch` and `router`. A tool with no switching machinery can support MoM in a day.
- **Level 1.** Level 0 plus all eight conditions, the evaluation model, holds, emergencies, safety limits, user override, the switch record, and demotion.
- **Level 2.** Level 1 plus `x-` conditions and the router.

A tool MUST document its level. A tool MUST NOT act on a manifest whose `mom` version has a major version it does not support; it MUST tell the user instead.

Validation and runtime are different postures. The published schema validates a file against this version: it flags unknown keys and keywords so authors catch typos. At runtime a tool MUST be liberal across minor versions: an unknown condition keyword makes the rule inert (6.3), an unknown model entry key that is not `x-` prefixed makes that requirement unevaluable and the tool MUST warn and ignore it, and an unknown top-level key from a newer minor version MUST be ignored with a warning. Rejection is reserved for files that are unparseable or whose `mom` major version is unsupported.

9 Security considerations

A manifest is project data and often arrives with cloned repositories. It cannot run code, but it steers two things that matter: where the conversation goes and what it costs.

- **Routing is confined to the user's own providers.** Resolution only ever selects accessible models: providers the user configured, with the user's own keys. A manifest MUST NOT cause a tool to send conversation content to a provider the user has not set up; a `prefer` pin naming an unconfigured provider is skipped, never honored. This is the line that stops a hostile repo from routing your session to an attacker's endpoint.

- **First use of a project manifest is a trust decision.** A tool MUST present a newly appeared or changed project manifest before the first model call it causes: at least the model entries and any spending posture (`power: max` entries, `spent-over` thresholds), the way it treats other repo-supplied configuration.
- **Spend steering.** `power: max` entries and aggressive rules can push a session onto expensive models. Tools SHOULD let users cap spend independently of any manifest, and the user override (7.4) always applies.
- **No secrets.** `x-` extension values MUST NOT carry credentials, and tools MUST NOT read secrets from the manifest.
- **Router privacy.** The router sends the turn's opening state (the user's message text only, never prior history) to a model; tools MUST apply the same privacy rules to router calls as to ordinary model calls.
- **Records are local.** Switch records contain session facts (cost, failure counts) and SHOULD follow the tool's existing log handling.

10 Examples

Non-normative. The same three files ship next to this spec in `specs/mom-examples/`.

`minimal.yaml` (one default, one escalation):

```
mom: "0.1"

models:
  everyday:
    power: medium
  thinker:
    power: max
    thinking: deep

start-with: everyday

switch:
  - when: stuck
    use: thinker
```

`cost-guard.yaml` (budget brake plus a long-conversation model):

```
mom: "0.1"
name: cost-guard

models:
  everyday:
    power: medium
    uses-tools: true
  thinker:
    power: max
    thinking: deep
    prefer:
      - anthropic/claude-opus-5
  marathon:
    power: medium
    memory: vast
```

```
    uses-tools: true

start-with: everyday

switch:
  - when: planning
    use: thinker
  - when: { chat-full: 85 }
    use: marathon
  - when: [stuck, looping, model-down]
    use: thinker
  - when: { spent-over: 5 }
    use: everyday
```

`review-pipeline.yaml` (a review flow with cheap proposer, strong verifier, and a tool-specific `x-` extension):

```
mom: "0.1"
name: review-pipeline

models:
  proposer:
    power: low
    settings: { temperature: 0 }
    prefer:
      - deepseek/deepseek-v4-flash
  verifier:
    power: max
    thinking: deep
    settings: { temperature: 0 }
    prefer:
      - anthropic/claude-sonnet-5
  judge:
    power: low
    settings: { temperature: 0 }
    x-mytool:
      role: collector

start-with: proposer

switch:
  - when: [stuck, looping]
    use: verifier
  - when: { x-mytool-stage: verify }
    use: verifier
  - when: { x-mytool-stage: judge }
    use: judge
```

A Conformance checklist (non-normative)

Level 0:

- ☐ finds `mom.yaml` in the documented order, project beating personal, no merging
- ☐ rejects unparseable files loudly and falls back to defaults

☐ validates against the schema when authoring; stays liberal at runtime per section 8

☐ resolves `start-with` through pins, then cheapest-fit catalog match, access-relative `power`, offline snapshot

☐ reports which requirement failed when an entry cannot resolve

☐ never routes to a provider the user has not configured

Level 1, additionally:

☐ implements all eight conditions with the exact meanings in 6.3, including the `chat-full` measurement procedure in 7.1 and `stuck` counter resets on switch

☐ accepts both the bare-keyword and parameterized condition forms; drops inert conditions from lists

☐ runs the stateless evaluation model: emergency rules first, then first match, else hold, else router, else `start-with`

☐ honors `hold` without letting it outvote a matching rule; lets emergencies override holds and switch mid-turn

☐ never switches to a model the conversation cannot fit; at most two switches per turn

☐ carries the full conversation across every switch

☐ suspends the manifest when the user picks a model explicitly

☐ writes a complete switch record per switch, including skipped models

☐ demotes a model after two stuck/looping/model-down switches under one entry, visibly, locally, and never into the manifest

☐ counts sub-work, router calls, and every manifest-caused call in `spent-over` and `tokens-over`

Level 2, additionally:

☐ treats unknown condition keywords as inert, reported once

☐ implements the router per the 6.4 contract: declared entries only, falls back to `start-with` on any failure, costs counted, picks recorded and held

B Design decisions (non-normative)

- **Plain words, not model IDs.** Pinned IDs go stale in months; routers transfer across model swaps (RouteLLM, arXiv 2406.18665). Requirements resolved against a live catalog keep a shared file working after the model landscape moves. Pins stay available as wishes for power users.
- **A reserved filename, not a file extension.** The identity of the format lives in the name `mom.yaml`, the way `package.json` and `AGENTS.md` work, not in an invented extension like `.mom`. A custom extension forfeits everything the ecosystem gives a `.yaml` file for free: syntax highlighting, linters, CI checkers, and schema-driven validation and autocomplete in every major editor via SchemaStore's filename matching. Cursor's `.mdc` is the cautionary tale: one proprietary extension, permanent tooling friction. Registering `mom.yaml` with SchemaStore belongs in the first public release of this spec.

- **Keywords with parameters, not sentences.** English phrases as values need parsing, break for non-English speakers, and drift between tools. The keyword-plus-parameter form is the shape of GitHub Actions events, CSS media queries, and alert rules: one small vocabulary, one testable meaning per keyword, thresholds as plain numbers.
- **Stateless evaluation.** Rules describe states, not transitions, for the same reason CSS media queries do: a rule that fires when a window narrows must unfire when it widens, or every trigger is a one-way ratchet. The first draft of this spec had the ratchet bug ("no match keeps the current model"); the worked trace in 7.1 is what replaced it.
- **Rules live in the tool, not a server.** Gateways see HTTP status and queue depth; only the client sees planning mode, chat fullness, failure streaks, and session spend. AgentOpt (arXiv 2604.06296) measured a 13–32x cost spread between good and bad model assignments and argued the decision belongs client-side.
- **No semantic guessing in conditions.** Keyword and free-text task classifiers cannot enumerate English and rot in practice. Every condition here is mechanically testable; anything fuzzier is the router's job, and the router is opt-in.
- **No quality grades in the manifest.** A field like `tool-calling: excellent` reads well and rots instantly: quality is a benchmark ranking that reshuffles with every model release, and two tools would grade it differently. Instead the protocol splits knowledge by who can keep it true: the file carries wishes (`prefer`), the catalog carries the market (benchmarks, prices), and local evidence carries what actually happened (7.6). The same reasoning applies to skills: how well a model follows a procedure is observed, not declared, and demotion turns the observation into behavior.
- **One bundle, switched atomically.** Model choice and thinking budget are one decision (UniScale, ICML 2026), so a model entry carries power, memory, thinking, and settings together.
- **Hold, safety limits, and the audit record are mandatory.** Flip-flopping and opaque switching are what make people turn auto-switching off; all three defenses are normative, not optional.
- **No savings claims.** Routing headroom is often an evaluation artifact (arXiv 2605.07395). The protocol promises control and explainability; cost numbers wait for step-level evaluation in the TwinRouterBench style (arXiv 2605.18859).

C Relationship to adjacent specs (non-normative)

- **Agent Plugins 1.0.0** standardizes agent packaging with no model field and no condition slot; a `mom.yaml` can travel inside a plugin via its `extensions` map.
- **MCP `modelPreferences`** declares per-request model hints and priorities, server to client, with no conditions and no persistence. MoM covers the other direction: a persistent, user-authored file the client itself honors. The two compose.
- **AGENTS.md and Agent Skills** conditionally attach instructions and procedures; neither can name a model. MoM deliberately lives in the same file-in-repo family, though its discovery is simpler on purpose: fixed paths, no walking up parent directories, no merging (section 5).

- **Existing switching.** Codex CLI profiles are bundles a typed flag activates; Cursor rules are conditions that only attach instructions; Claude Code's opusplan and fallbackModel, Goose's lead/worker split, and Copilot Auto switch automatically under rules the user cannot read or edit. MoM is the open join of the three columns: user-authorable conditions, bound to whole model setups, portable across tools.

D Anticipated objections (non-normative)

Answers to the criticisms this spec expects, so they are argued here rather than discovered later.

- **"The same file picks different models on different tools, so portability is fake."** Portability here means the same *policy*, not the same bytes on the wire: switch to something smarter when planning or stuck, something cheaper past a budget. `power` is access-relative on purpose; a file that named exact models would be dead in a quarter and useless to anyone with different keys. Users who want exact models have `prefer`, and `mom check` shows the actual resolution.
- **"Claude Code already does this."** It switches under six hardcoded behaviors the user cannot read or edit, and none of them travel. The point is not that switching exists; it is who gets to write the rules and whether the file works anywhere else.
- **"Why not just one smart router instead of rules?"** Routers are opaque, cost a call, and cannot be diffed in review. Rules are deterministic, auditable, and shareable; the router exists as an opt-in fallback for the genuinely fuzzy cases, and it never overrides a rule.
- **"Cost estimates differ between tools, so `spent-over: 5` is not portable."** They differ, and the spec does not pretend otherwise: cache pricing, reasoning tokens, and provider markup all drift. That is why section 6.3 defines the minimum cost model and makes tools disclose which components they cannot price. The condition is a budget tripwire, not an invoice; a tool that cannot estimate at all must say so, not guess.
- **"Percent-of-window rules flap."** Measured naively, yes; that is why `chat-full` is measured against the model the evaluation would otherwise select, holds exist, and switches are capped at two per turn. The flap analysis is in 6.3, the measurement procedure in 7.1, and the caps in 7.3, not left to implementors.
- **"A repo file that spends my money is a supply-chain hole."** Resolution is confined to providers the user configured, pins to unknown providers are skipped, first use is a trust decision, and the user's explicit choice suspends the file. The manifest can express policy only over models the user could already reach.
- **"`stuck` and `looping` detection differ per tool."** `stuck` is mechanical: counted failed tool steps, a shared threshold, shared reset rules. Only `looping` detection is tool-defined, and it is a separate keyword precisely so the fuzzy part is fenced off. Two tools may notice a loop a turn apart; they must respond to it identically.
- **"Why YAML?"** Because the target author is a person with an editor, the adjacent files (AGENTS.md, skills, CI configs) set the expectation, and the format is one schema away from strict validation. JSON is machine-first; TOML nests badly for rule lists.
- **"Turn boundaries are too coarse; a bad model wastes a whole turn."** The two emergencies exist precisely for the mid-turn dead ends, and everything else compounds

slowly enough that a one-turn delay costs cents. Sub-turn switching would need step-level state that no two tools share yet; TwinRouterBench (arXiv 2605.18859) exists because the field is only now defining it. 0.1 stays coarse on purpose.

- **"Nobody will implement it."** Level 0 is a parser and a default, a day of work, and every harness listed in Appendix C already has the switching machinery Level 1 needs. The spec ships with a schema, test scenarios, and a conformance checklist; adoption is a bet, but not an unpriced one.